

Carry / Levered LP Vault — Contract Overview

Grant and audit reference for `LeveredLpVault` on **Robinhood Chain Testnet (chain id 46630)**.

Primary vault: `0x72A053120f03B10c5506d2325f92F59B0FfC36c8` (Sourcify verified)

1. Overview

Carry pairs a stock-token holder (**junior**) with a USDG lender (**senior**) into a **50/50 book**: junior posts MSTR, senior posts matching USDG. Equity tracks the stock while the book accrues LP fees over a fixed term. This vault is the **custody and accounting layer**.

- Junior deposits MSTR → **Open** until USDG matches, then **Active** for **7 days**.
- **4%** is the **pool-floor pace** for early-exit coupon math — not an extra tip on top of the maturity borrow fee.
- At **maturity**, seniors are owed principal + a **borrow fee** (`borrowAprWad`, capped at 5%, $\times$ term / year).
- **Early exit** closes the whole position: fees cover `principal  $\times$  4%  $\times$  elapsed/365` when available; otherwise junior MSTR from the position covers the gap.
- Lenders withdraw **idle** USDG anytime; **matched** USDG unlocks via replacement liquidity or position close.

Contract snapshot

Item	Value
Contract	<code>LeveredLpVault</code> (non-upgradeable)
Tokens / oracle	Immutable <code>mstr</code> , <code>usdg</code> ; immutable <code>IPriceOracle</code>
Term / borrow APR	Immutable <code>term</code> $\leq$ 7 days; <code>borrowAprWad</code> $\leq$ 5% WAD
Protocol cut	Immutable <code>protocolCutWad</code> $\leq$ 20% of each fee
Early-exit APR	Constant <code>EARLY_EXIT_APR_WAD</code> = <code>0.04e18</code>
Pause / reentrancy	Owner pause gates deposits; <code>nonReentrant</code> on value paths
External LP	No <code>PoolManager</code> approvals; <code>joinPool</code> reverts

Inventory is held inside the vault. `mark()` reports 2 $\times$ equity vs an unlevered 50/50 $\sqrt{p}$ comparison without moving inventory.

2. Actors

Actor	Role	Key actions
Junior (stock depositor)	Posts MSTR	<code>depositJunior</code> , <code>withdrawUnmatched</code> , <code>earlyExit</code> ; receives leftover MSTR/fees on settle
Senior (lender)	Posts USDG	<code>depositSenior</code> , <code>withdrawSenior</code> (idle + claims)
Owner / protocol	Pause + rescue key	<code>pause</code> , <code>unpause</code> , <code>rescueToken</code> (non-principal only). Cannot set <code>oracle/term/APR/cut</code> ; cannot pull MSTR/USDG
Fee source	Accrues LP fees	<code>accrueLpFee</code> / <code>MockFeePool.payFee</code> pushes USDG into the vault
Anyone	Keepers	<code>settle</code> after term (proceeds go to junior + seniors, not the caller)

3. Lifecycle

1. **Deposit stock** — `depositJunior(mstrAmount)`. If no idle USDG → Open queue. If idle USDG $\geq$ oracle notional → match immediately (Active, `openedAt` = now).
2. **Lend USDG** — `depositSenior(amount)`. Credits senior principal; FIFO-matches Open juniors; each match sets `seniorPrincipal`, `entryPriceWad`, `openedAt`, increases `reservedSenior`.
3. **Active** — Position earns when matched. Timer = `term` from match.
4. **Fee accrual** — Caller pushes USDG via `accrueLpFee`. `protocolCutWad` → backstop; remainder → `position.feeUsdg`.
5. **Early exit** (junior, before term) — Coupon $\text{principal} \times 0.04 \times \text{elapsed}/365$. Fees first; gap → sell MSTR from position; return remaining shares. Releases `reservedSenior`.
6. **Maturity settle** (after term) — Borrow fee $\text{principal} \times \text{borrowApr} \times \text{term}/365$. Waterfall: position fees → backstop → minimum MSTR. Junior gets leftover MSTR + leftover fees.
7. **Lender replace / exit** — `freePrincipal(senior) = seniorPrincipal \times freeSenior / totalSeniorPrincipal`. Extra senior deposits raise idle capacity. Matched size unlocks on early exit/settle when `reservedSenior` drops.

4. Function reference

Admin

Function	Who	Preconditions	Effects / funds
<code>pause()</code>	owner	—	<code>paused = true</code> ; new deposits revert
<code>unpause()</code>	owner	—	<code>paused = false</code>
<code>rescueToken(token,to,amount)</code>	owner	<code>token ∉ {mstr,usdg}</code>	Transfers non-principal token

Junior

Function	Who	Preconditions	Effects / funds
<code>depositJunior(mstrAmount)</code>	anyone (when unpaused)	<code>amount > 0</code>	Pulls exact MSTR; enqueues Open; tries match
<code>withdrawUnmatched(positionId)</code>	position owner	Open, not settled	Returns all MSTR; no coupon
<code>earlyExit(positionId)</code>	position owner	Matched, before term, not settled	Closes position; coupon from fees; gap via MSTR from position; pushes leftover MSTR/USDG to junior

Senior

Function	Who	Preconditions	Effects / funds
<code>depositSenior(amount)</code>	anyone (when unpaused)	<code>amount > 0</code>	Pulls exact USDG; increases principal; FIFO match
<code>withdrawSenior(principalAmount)</code>	senior	<code>principalAmount ≤ freePrincipal</code>	Returns idle principal + claimable yield USDG + claimable MSTR

Fees / backstop

Function	Who	Preconditions	Effects / funds
<code>accrueLpFee(positionId, amount)</code>	anyone	Matched, not settled	Pulls exact USDG; cut → backstop; rest → position
<code>fundBackstop(amount)</code>	anyone	<code>amount > 0</code>	Pulls USDG into backstop

Settlement / views

Function	Who	Preconditions	Effects / funds
<code>settle(positionId)</code>	anyone	Matched, term elapsed, not settled	Maturity waterfall; pushes to junior; credits senior accumulators
<code>previewEarlyExit / previewEarlyExitCoupon</code>	view	—	Coupon + share sale preview
<code>previewBorrowFee / previewSeniorAssets / previewMstrToCover</code>	view	—	Quoting helpers
<code>freeSenior / freePrincipal / isMatched / mark</code>	view	—	Capacity / MTM
<code>joinPool(bytes)</code>	anyone	—	Reverts <code>ExternalLpForbidden</code>

Constructor immutables: `mstr`, `usdg`, `oracle`, `term`, `borrowAprWad`, `protocolCutWad`, `owner`.

5. Accounting

What “whole” means

If fees accrue at the **4% floor** on LP notional for the week, then after the **20% protocol cut**, remaining fees cover the senior **borrow fee** so:

- Junior can withdraw **all shares** (and leftover fees), and
- Senior receives **principal + borrow fee** in USDG (or MSTR if the fee waterfall sold shares).

4% is **not** an extra tip to the lender on top of the borrow fee.

Early-exit coupon

```
couponOwed = seniorPrincipal * EARLY_EXIT_APR_WAD * elapsed / (WAD * YEAR)
             = principal * 0.04 * elapsed_seconds / (365 days)
```

Waterfall:

1. `fromPositionFees = min(position.feeUsdg, couponOwed)`
2. `usdgGap = couponOwed - fromPositionFees`
3. If `gap > 0`: `mstrSold = ceil(gap / price)` from **this position's MSTR** (credited to seniors)
4. Junior receives `mstrAmount - mstrSold` and any leftover fees

No external USDG pull from the junior's wallet on early exit.

Maturity borrow fee

```
borrowFeeOwed = seniorPrincipal * borrowAprWad * term / (WAD * YEAR)
```

Deploy constants: borrow APR **5%**, term **7 days**, protocol cut **20%**.

Waterfall: position fees → backstop → minimum junior MSTR. Leftover fees go to junior as `juniorYield`.

Senior claims

On settle/early exit, USDG paid toward fees increases `accYieldPerPrincipal`; sold MSTR increases `accMstrPerPrincipal`. Seniors harvest via `withdrawSenior`.

6. Security model

Control	Behavior
Pause	Blocks <code>depositJunior</code> / <code>depositSenior</code> only. Exit, settle, and withdraw remain available
Owner powers	Pause/unpause + rescue of non-principal tokens
Owner cannot	Withdraw MSTR/USDG, change oracle/term/APR/cut, or pull principal via rescue
Reentrancy	<code>nonReentrant</code> on deposits, withdraws, fees, early exit, settle, rescue
CEI	State + events updated before external token pushes on exit paths
Fee-on-transfer	Rejected (<code>FeeOnTransfer</code>) on pulls
External LP	<code>joinPool</code> forbidden; no <code>PoolManager</code> allowance
Oracle	Immutable oracle address on the vault

Invariants

1. `reservedSenior` $\leq$ `totalSeniorPrincipal`
2. Settled positions never pay twice (`settled` latch)
3. Owner rescue cannot touch `mstr` / `usdg`
4. Matched MSTR leaves only via junior payout or senior MSTR claims after sale-to-cover
5. No external `PoolManager` approvals from the vault

7. Threat notes

Attack / misuse	Mitigation	Test
Random drains senior/junior funds	No open withdraw; settle pays owners not caller	<code>test_randomAddressCannotPullFunds</code>
Owner steals principal via rescue	<code>PrincipalToken</code> revert	<code>test_ownerCannotStealPrincipal</code>
Double early exit / settle	<code>AlreadySettled</code>	<code>test_earlyExitCannotDoubleExit</code>
Attacker early-exits someone else	<code>NotJunior</code>	<code>test_earlyExitOnlyJunior</code>
Early exit after term	<code>TermElapsed</code> ; must settle	<code>test_earlyExitBlockedAfterTerm</code>
Fee on unmatched position	<code>NotMatched</code>	<code>test_accrueFeesRequiresMatched</code>
Deposit while paused	<code>Paused</code>	<code>test_pauseBlocksNewDeposits</code>
External LP beside vault	<code>ExternalLpForbidden</code>	<code>test_externalCannotLpBesideVault</code>
Lender pulls matched USDG	<code>InsufficientFree</code> until replace/close	<code>test_lenderReplaceUnlocksExit</code>
Reentrancy on token callbacks	<code>nonReentrant</code>	Covered by modifier on value paths
Fee-on-transfer silent credit	Balance delta check	<code>FeeOnTransfer</code> in <code>_pullExact</code>
Wrong-chain broadcast	Deploy script refuses chain ids 4663 and 1	<code>DeployCarryTestnet</code> guards

8. Carry UI → contract mapping

Carry screen / control	Vault call	Notes
Deposit MSTR / Deposit {sym}	<code>mstr.approve → depositJunior(amount)</code>	Pause-gated; Open if no USDG
Position badge Open	<code>!isMatched(id)</code>	Unmatched; no fees yet
Position badge Active	<code>isMatched(id)</code>	7-day timer from <code>openedAt</code>
Lend / deposit USDG (lender)	<code>usdg.approve → depositSenior(amount)</code>	FIFO-matches Open queue
Active position view	<code>positions(id), mark(id), previewEarlyExit(id)</code>	Read-only
Withdraw Early (stock)	<code>earlyExit(id)</code>	Junior only; fees then MSTR from position
Withdraw (Open / unmatched stock)	<code>withdrawUnmatched(id)</code>	Full MSTR back, no coupon
Withdraw after 7d / settle	<code>settle(id)</code> then junior already paid	Anyone can call settle
Lender withdraw idle	<code>withdrawSenior(freePrincipal(me))</code>	Available when idle > 0
Lender withdraw matched	More <code>depositSenior</code> (replace) or wait for close	Else <code>InsufficientFree</code>
Fee / rewards display	Driven by <code>accrueLpFee</code>	USDG pushed into position fees

9. Testnet addresses (Robinhood Chain Testnet 46630)

Item	Value
Network	Robinhood Chain Testnet
Chain ID	46630
RPC	https://rpc.testnet.chain.robinhood.com
Explorer	https://explorer.testnet.chain.robinhood.com

Primary vault

Name	Address	Explorer
LeveredLpVault	0x72A053120f03B10c5506d2325f92F59B0FfC36c8	verified

Supporting contracts

Name	Address	Explorer
mockMstr (stock token)	0x762019309B536bbb89577422FaaFBeC9659f8728	verified
mockUsdg	0x25030Bff74764aD72b912276a603717DB1C00644	verified
mockOracle (\$100 WAD)	0xc74Af7E23A2B4b46B5Fe05E5c7c5ec0BB5dbc5B7	verified
mockFeePool	0xa4122524aD97Aab05cAC7F99c04Cd060D02F0771	verified
vaultOwner	0xcA44F2dbB2D43b93b39F01B88E0f0e2966983c57	view

Deploy constants: term **7 days**, borrow APR **5%**, protocol cut **20%**, oracle **\$100** WAD.

Source paths

Path	Role
<code>contracts/src/LeveredLpVault.sol</code>	Vault
<code>contracts/src/mocks/MockERC20.sol</code>	Testnet tokens
<code>contracts/src/mocks/MockOracle.sol</code>	Testnet price feed
<code>contracts/src/mocks/MockFeePool.sol</code>	Fee pusher
<code>contracts/test/LeveredLpVault.t.sol</code>	Foundry tests
<code>contracts/script/DeployCarryTestnet.s.sol</code>	RH testnet 46630 deploy
<code>CARRY_CONTRACT.md</code>	Full contract pack (repo root)
<code>CARRY_TESTNET.md</code>	Deploy addresses + explorer links